Name: P Lakshmi Srinivasulu Reddy
Register Number: 192465051

## TASK 1 — Design the Algorithms 1.1

### Algorithm A: Iterative Linear Search

**Design strategy:** Brute force.

**Basic idea**

Linear Search examines the registration numbers one by one, starting at the first record. If the target registration number matches a record, its position is returned. If all records are examined without finding a match, the algorithm reports that the record does not exist.

The algorithm does not require sorting or any additional data structure. This makes it suitable for small datasets or records that are not maintained in a particular order.

**Pseudocode**

```
ALGORITHM LinearSearch(records, target)
n ← length(records)

  FOR i ← 0 TO n - 1 DO
    IF records[i].registrationNumber = target THEN
      RETURN i
    END IF
  END FOR

  RETURN -1
END ALGORITHM
```

**Preprocessing**

No preprocessing is required. The algorithm can operate directly on the existing dataset.

**Justification**

Linear Search is selected because it is simple, works on unsorted data and provides a baseline for comparing more advanced algorithms. However, its performance can deteriorate significantly as the number of records increases.

### 1.2 Algorithm B: Recursive Binary Search

**Design strategy:** Divide and conquer.

**Basic idea**

Binary Search requires records to be sorted by registration number. It compares the target with the middle record.

- If they match, the search succeeds.
- If the target is smaller, the search continues in the left half.
- If the target is larger, the search continues in the right half.

Each recursive call reduces the remaining search interval to approximately half its previous size.

**Pseudocode**

```
ALGORITHM RecursiveBinarySearch(records, target, low, high)


  IF low > high THEN
    RETURN -1
  END IF


  mid ← floor((low + high) / 2)


  IF records[mid].registrationNumber = target THEN
    RETURN mid
  END IF


  IF target < records[mid].registrationNumber THEN
RETURN RecursiveBinarySearch(
      records, target, low, mid - 1
    )
  ELSE
    RETURN RecursiveBinarySearch(
      records, target, mid + 1, high
    )
  END IF

END ALGORITHM
```

Initial call:

RecursiveBinarySearch(records, target, 0, n - 1)

**Preprocessing**

The records must be sorted by registration number. For an initially unsorted dataset, comparison-based sorting typically costs (O(n\log n)).

If the database is already maintained in sorted order, this sorting cost need not be repeated for every search. **Justification**

Recursive Binary Search is selected because it illustrates divide-and-conquer design and provides logarithmic search complexity. It is particularly useful when many searches are performed against a dataset that changes infrequently.

**1.3 Algorithm C: Hash-Table Search**

**Design strategy:** Hashing / direct key-based lookup.

**Basic idea**

A hash table stores each student record using the registration number as its key. A hash function maps the key to a table location, allowing the system to locate the corresponding record without scanning all records or repeatedly dividing the search interval.

If two keys map to the same location, a collision-resolution technique such as separate chaining or open addressing is required.

**Pseudocode**

```
ALGORITHM BuildHashTable(records)
   CREATE an empty hash table H

   FOR each record IN records DO
      H[record.registrationNumber] ← record
   END FOR

   RETURN H
END ALGORITHM
ALGORITHM HashSearch(H, target)
   IF target EXISTS IN H THEN
      RETURN H[target]
   ELSE
      RETURN NOT_FOUND
```

END IF

END ALGORITHM

**Preprocessing**

The hash table must be constructed from the student records. Assuming a suitable hash function and controlled load factor, construction takes expected (O(n)) time and requires (O(n)) additional space.

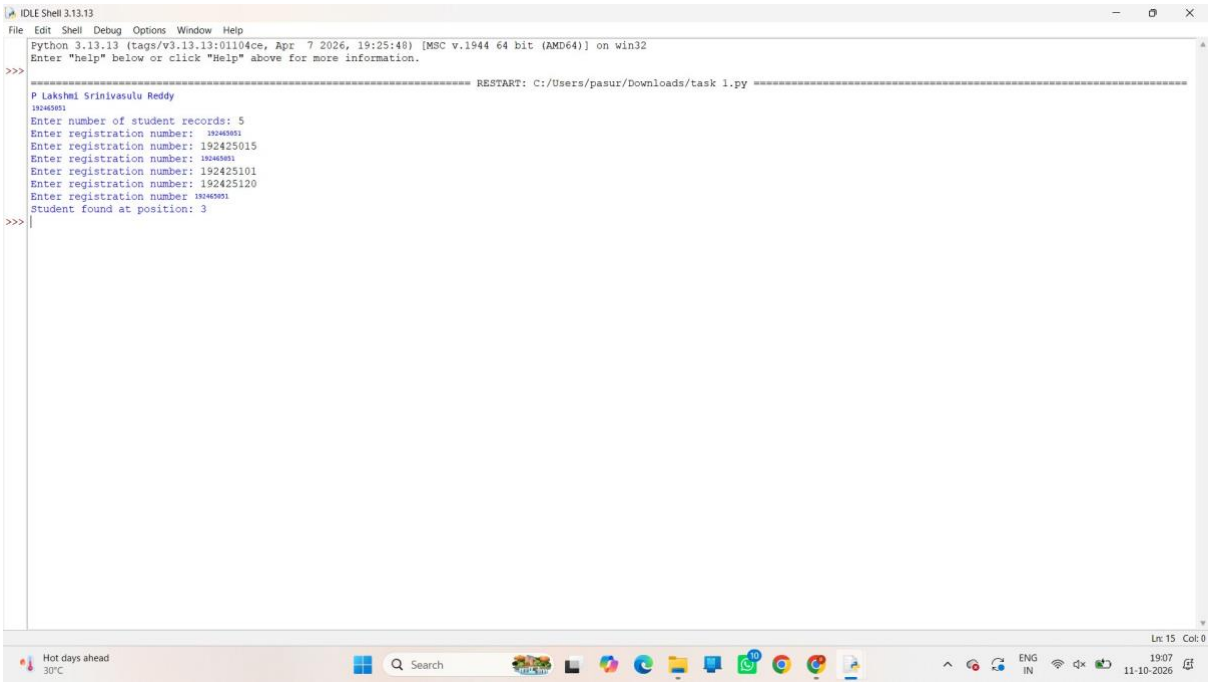
**Justification**

Hash-Table Search is selected because registration numbers are unique keys and the application requires frequent exact-match lookups. It is a strong candidate when the database contains many records but changes relatively infrequently.

Unlike Binary Search, hashing does not naturally maintain records in sorted order and does not efficiently support ordered range queries.

**Comparison of design strategies**

| **Property** | **Linear Search** | **Recursive Binary Search** | **Hash-Table Search** |
|---|---|---|---|
| Strategy | Sequential scanning | Divide and conquer | Hash-based lookup |
| Iterative/recursive | Iterative | Recursive | Iterative construction and lookup |
| Requires sorted data | No | Yes | No |
| Preprocessing | None | Sorting, if needed | Hash-table construction |
| Expected search cost | (O(n)) | (O(\log n)) | (O(1)) |
| Extra storage | (O(1)) | (O(\log n)) recursive stack | (O(n)) |
| Main limitation | Slow for large datasets | Requires sorted data | Collisions and additional memory |




**TASK 2 — Identify the Basic Operation.**

The basic operation is the operation whose repeated execution predominantly determines the running time of an algorithm.

**2.1 Linear Search**

**Basic operation:** Comparison of the target registration number with a record's registration number.

It occurs in the conditional statement:

IF records[i].registrationNumber = target

The number of comparisons depends on the target's position. A match at the first record requires one comparison, while an unsuccessful search requires examining every record.

Consequently, the comparison count can increase linearly with (n).

**2.2 Recursive Binary Search**

**Basic operation:** Comparison of the target with the middle record's registration number.

It occurs in the equality test and the subsequent ordering test. Each recursive call examines one middle element and reduces the remaining interval to approximately half its previous size.

The number of comparisons therefore increases logarithmically rather than linearly with the number of records.

For counting purposes, one may count each key comparison separately: one equality comparison and, when needed, one ordering comparison.

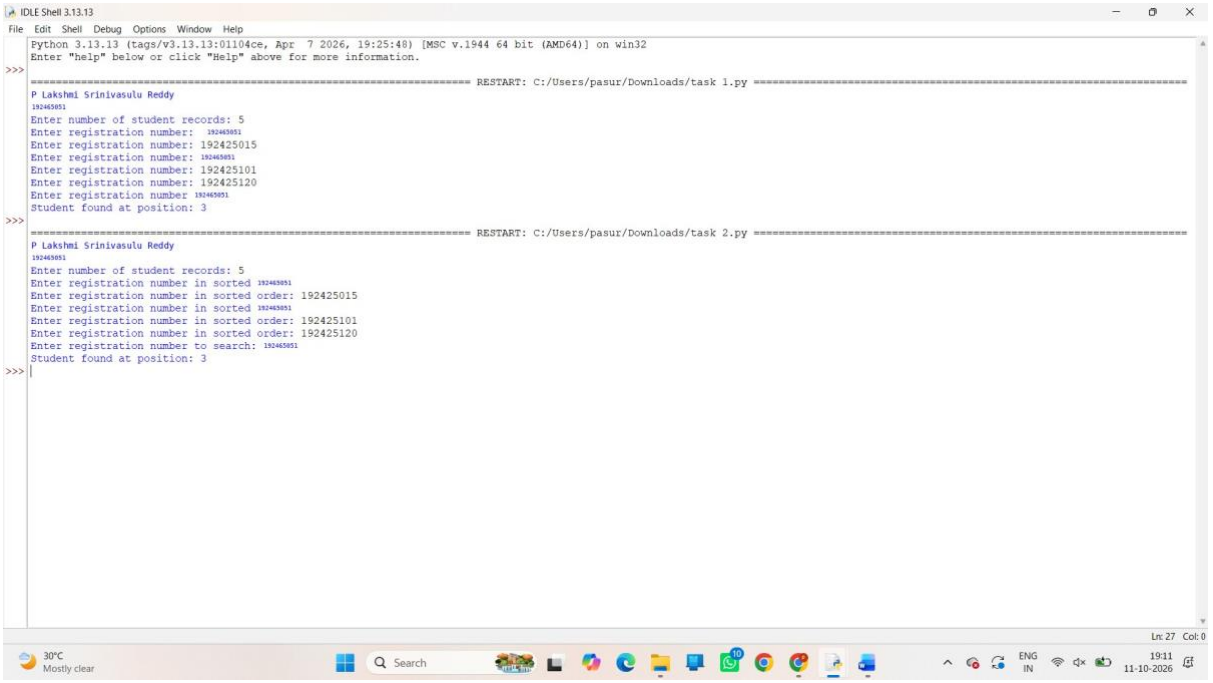
**2.3 Hash-Table Search**

**Basic operation:** Hash-key lookup, including the examination of a table entry or bucket and any necessary key comparison.

The hash function first determines the relevant location. The implementation then checks the location or follows the collision-resolution structure until the key is found or absence is established.

With a well-distributed hash function and a controlled load factor, the expected number of entries examined remains approximately constant as (n) increases. Under severe collisions, however, a lookup may examine a number of entries proportional to (n).

This operation is appropriate because it captures the work required to resolve the target key, rather than merely counting the hash-function invocation.




**TASK 3 — Derive the Time-Complexity Function Mathematically.**

### 3.1 Linear Search

Let (n) be the number of records.

Each iteration performs one key comparison. If the target is at zero-based position (k), the algorithm performs

[
C_L(n)=k+1.
]

For a successful search, (0\leq k<n). Therefore,

[
1\leq C_L(n)\leq n.
]

For an unsuccessful search, every record must be examined:

[
C_L(n)=n.

]

The running-time function can be represented as

[
T_L(n)=aC_L(n)+b,
]

where (a) is the constant cost of a comparison and (b) represents fixed overhead. In

the worst case,

[
T_L(n)=an+b.
]

Thus,

[
\boxed{T_L(n)=\Theta(n)}
]

in the worst case.

No preprocessing is required. For (q) independent searches, the worst-case total cost is

[
\boxed{T_{\text{total}}(n,q)=\Theta(qn)}.
]

**3.2 Recursive Binary Search**

Let (n) be the number of elements in the current search interval.

Each call performs a constant amount of work: it calculates the midpoint and executes at most two key comparisons. It then continues in only one half of the interval. Ignoring integer-rounding details initially, the recurrence is

[
T_B(n)=T_B(n/2)+c,
]

where (c) is constant work per call.

After one recursive reduction, the interval size is (n/2). After two reductions, it is (n/4). After (k) reductions, it is approximately

[
\frac{n}{2^k}.
]

The search reaches an interval of size at most one when

[
\frac{n}{2^k}\leq 1.

]

Rearranging,

[
n\leq 2^k
]

and hence

[
k\geq\log_2 n.
]

Therefore, the number of recursive levels is proportional to (\log_2 n), giving

[
\boxed{T_B(n)=\Theta(\log n)}.
]

**Preprocessing and total cost**

If the original dataset is unsorted, comparison-based sorting typically costs

[
T_{\text{sort}}(n)=\Theta(n\log n).
]

One search then costs (\Theta(\log n)). For (q) searches against the same sorted dataset,

[
T_{\text{total}}(n,q)
=\Theta(n\log n)+\Theta(q\log n).
]

Therefore,

[
\boxed{T_{\text{total}}(n,q)=\Theta(n\log n+q\log n)}.
]

If the data is already sorted, the sorting term is omitted.

**3.3 Hash-Table Search**

Let (n) be the number of records and (m) the number of hash-table slots. The load factor is

[
\alpha=\frac{n}{m}.
]

The table is built by processing each record and inserting its key. With an appropriate hash function and resizing policy, the expected insertion cost is constant per record. Thus, the expected construction cost is

$$
T_{\text{build}}(n)
=\sum_{i=1}^{n}\Theta(1) =\Theta(n).
$$

During a lookup, the hash function identifies a bucket or slot. If collisions are well distributed, the expected amount of collision-resolution work is bounded by a constant when the load factor is controlled.

Consequently, expected lookup cost is

$$
\boxed{T_H(n)=\Theta(1)}.
$$

For (q) lookups, the expected combined construction and search cost is

$$
T_{\text{total}}(n,q)
=\Theta(n)+\Theta(q).
$$

Hence,

$$
\boxed{T_{\text{total}}(n,q)=\Theta(n+q)}
$$

under the expected-time hashing assumptions.

In the worst case, many keys may collide into the same chain or probe sequence. A lookup can then take (\Theta(n)), making the worst-case combined cost

$$
\boxed{T_{\text{worst}}(n,q)=\Theta(n+qn)}.
$$

This worst case is not the expected behaviour of a properly implemented hash table, but it must be acknowledged in a rigorous analysis.

**Summary of preprocessing and search costs**

| **Algorithm** | **Preprocessing** | **(q) searches after One search preprocessing** | |
|---|---|---|---|
| Linear Search | None | (O(n)) worst case | (O(qn)) |
| Binary Search | (O(n\log n)) if sorting is needed | (O(\log n)) | (O(n\log n+q\log n)) |
| Hash-Table Search | (O(n)) expected | (O(1)) expected | (O(n+q)) expected |

The preprocessing term is essential. Comparing only the time of one search would omit the cost of sorting or constructing the index.

**TASK 4 — Best-, Average- and Worst-Case Analysis.**

**4.1 Linear Search**

**Best case:** The target is the first record. Only one comparison is required:

$$
C_{\text{best}}(n)=1.
$$

**Average case:** Assume that a successful target is equally likely to occur at any of the (n) positions. The expected number of comparisons is

$$
C_{\text{avg}}(n)
=\frac{1+2+\cdots+n}{n}.
$$

Using the arithmetic-series formula,

$$
C_{\text{avg}}(n)
=\frac{n(n+1)}{2n} =\frac{n+1}{2}.
$$

Therefore, the average successful search requires approximately (n/2) comparisons. An unsuccessful search requires exactly (n) comparisons.

**Worst case:** The target is the last record or is absent:

$$
C_{\text{worst}}(n)=n.
$$

The performance degrades because the algorithm has no mechanism for eliminating large portions of the dataset.

**4.2 Recursive Binary Search**

**Best case:** The target is the middle element of the initial interval. One equality comparison finds the target:

$$
C_{\text{best}}(n)=1.
$$

**Average case:** For a successful search with targets distributed across the sorted records, the number of comparisons depends on the target's depth in the implicit binary-search tree. The average is logarithmic:

[

C_{\text{avg}}(n)=\Theta(\log n).
]

**Worst case:** The target is absent or lies at a position requiring the deepest search path. The interval is repeatedly halved until it becomes empty or the target is found.

The maximum number of midpoint examinations is bounded by approximately

[
C_{\text{worst}}(n)
\leq \lfloor\log_2 n\rfloor+1.
]

This explains why Binary Search remains efficient for large datasets: each comparison eliminates roughly half the remaining candidates.

### 4.3 Hash-Table Search

**Best case:** The target maps to an available slot or bucket and is found immediately. The lookup takes constant time:

[
T_{\text{best}}(n)=\Theta(1).
]

**Average case:** With a suitable hash function, a controlled load factor and effective collision resolution, the expected number of entries examined is constant:

[
T_{\text{avg}}(n)=\Theta(1).
]

For separate chaining, expected lookup cost depends on the load factor (\alpha=n/m). Keeping (\alpha) bounded helps maintain efficient lookups.

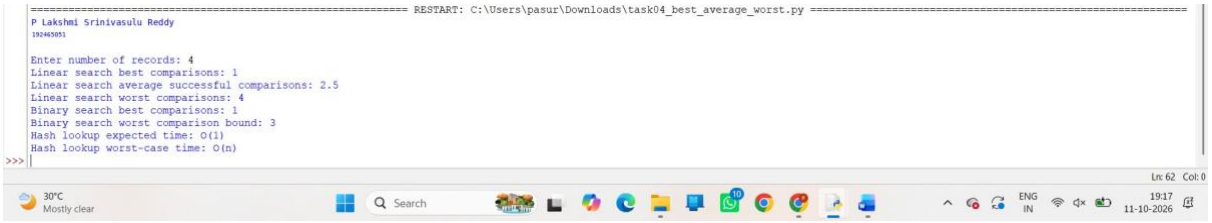
**Worst case:** Many keys collide into the same chain or probe sequence. The algorithm may have to inspect a linear number of entries:

[
T_{\text{worst}}(n)=\Theta(n).
]

Therefore, the practical performance of hashing depends on the quality of the hash function, collision resolution, resizing policy and key distribution.

### Comparative interpretation

Linear Search has a particularly poor worst case because it may examine every record. Binary Search guarantees logarithmic search depth on sorted data. Hashing usually provides the fastest exact-match lookup, but unlike Binary Search, it does not guarantee constant-time performance for every possible key distribution.




**TASK 5 — Perform Asymptotic Analysis.**

Asymptotic analysis describes how an algorithm's resource requirements grow as the input size increases.

- (O(f(n))) gives an asymptotic upper bound.
- (\Omega(f(n))) gives an asymptotic lower bound.
- (\Theta(f(n))) gives a tight asymptotic bound when both upper and lower bounds have the same order of growth.

**5.1 Linear Search**

In the worst case, Linear Search performs (n) comparisons. Its worst-case time complexity is

[
T_L(n)=\Theta(n).
]

Its best-case time is (\Theta(1)), while its worst-case time is (O(n)). For uniformly distributed successful searches, its average comparison count is ((n+1)/2), giving (\Theta(n)) average time.

If the number of records increases from 1,000 to 1,000,000, the worst-case comparison count increases by a factor of 1,000.

**5.2 Recursive Binary Search**

Each recursive call reduces the search interval by approximately half:

[
T_B(n)=T_B(n/2)+\Theta(1).
]

The resulting complexity is

[
T_B(n)=\Theta(\log n).
]

The best case is (\Theta(1)), while the worst and average cases are (\Theta(\log n)) under the standard search assumptions.

For one million records, the maximum number of midpoint examinations is approximately 20. This is a substantial reduction from the million comparisons potentially required by Linear Search.

### 5.3 Hash-Table Search

Under standard assumptions concerning hash distribution and load factor, expected lookup time is

[
T_H(n)=\Theta(1).
]

Its best-case and expected lookup times are constant, but its worst-case lookup time is (O(n)) because of collisions.

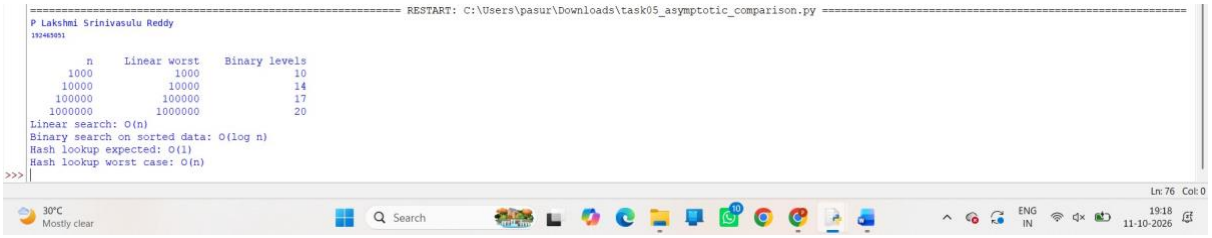
The table must first be constructed, which costs expected (\Theta(n)) time and (\Theta(n)) additional space. Therefore, constant-time lookup should not be confused with constant-time construction.

### 5.4 Growth comparison

| **Records** | **Linear worst-case** | **Binary maximum midpoint** | **Hash lookup** |
|---|---|---|---|
| **(n)** | **comparisons** | **examinations** | |
| 1,000 | 1,000 | 10 | Expected constant |
| 10,000 | 10,000 | 14 | Expected constant |
| 100,000 | 100,000 | 17 | Expected constant |
| 1,000,000 | 1,000,000 | 20 | Expected constant |

The hash column describes expected lookup behaviour, not an exact comparison count.

The table demonstrates the practical significance of asymptotic growth. Linear Search becomes increasingly expensive as records accumulate. Binary Search grows slowly because each comparison discards half the search interval. Hash-Table Search generally avoids a search cost proportional to the total number of records, although it requires additional memory and index construction.




**TASK 6 — Formulate the Recurrence Relation.**

The recursive algorithm selected for recurrence analysis is Recursive Binary Search.

Let (n) denote the number of records in the current search interval.

**6.1 Work performed by one recursive call** Each

call performs the following operations:

1. Checks whether the interval is empty.
2. Calculates the midpoint.
3. Compares the target with the midpoint registration number.
4. Chooses at most one half of the interval for the next recursive call.

The work outside the recursive call is constant, denoted by (c).

**6.2 Size of the recursive subproblem**

After the midpoint is examined, the remaining interval contains at most approximately half the original elements. The algorithm makes only one recursive call, not two. The recurrence is therefore

[
T(n)=T(\lfloor n/2\rfloor)+c, \qquad
n>1.
]

**6.3 Base case**

When the interval contains zero or one candidate, the algorithm terminates after constant work. For an empty interval, the target is absent; for a one-element interval, the record is either found or rejected.

We can represent the base case as

[
T(1)=d,
]

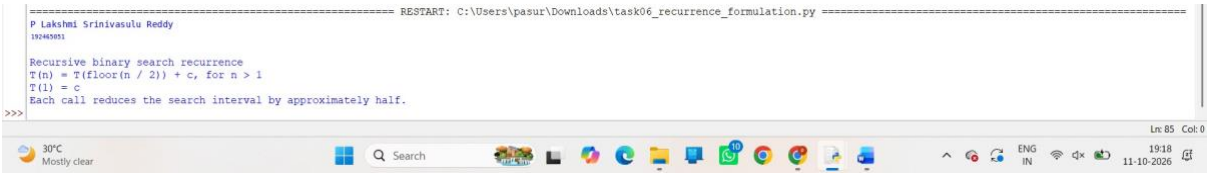
where (d) is a constant. An implementation may also use an empty-interval base case (T(0)=d_0).

### 6.4 Final recurrence

Ignoring floor effects for the purpose of asymptotic analysis,

$$
\boxed{
T(n)=
\begin{cases} d, & n\leq 1,\\
T(n/2)+c, & n>1.
\end{cases}
}
$$

This recurrence follows directly from the algorithm: constant work is performed per call, and only one subproblem of approximately half the original size is processed.




### TASK 7 — Solve the Recurrence Relation.

The recurrence obtained in Task 6 is

$$
T(n)=T(n/2)+c, \qquad T(1)=d.
$$

The iteration method is suitable because the recurrence contains one subproblem whose size decreases geometrically.

**Step 1: Expand the recurrence**

Starting with

$$
T(n)=T(n/2)+c,
$$

expand the smaller problem:

$$
T(n/2)=T(n/4)+c.
$$

Substituting gives

$$
T(n)=T(n/4)+2c.
$$

Expanding again,

$$
T(n)=T(n/8)+3c.
$$

After (k) expansions,

$$
T(n)=T(n/2^k)+kc.
$$

**Step 2: Determine the number of expansions**

The recursion reaches its base case when

$$
\frac{n}{2^k}\leq 1.
$$

Therefore,

$$
n\leq 2^k.
$$

Taking logarithms to base 2,

$$
k\geq\log_2 n.
$$

Thus, the number of expansions is proportional to (\log_2 n).

**Step 3: Substitute the base case**

At termination, (T(n/2^k)=d). Therefore,

$$
T(n)=d+kc.
$$

Since (k) is proportional to (\log_2 n),

$$
T(n)=d+c\log_2 n.
$$

Consequently,

$$
\boxed{T(n)=\Theta(\log n)}.
$$

**Interpretation**

The recurrence confirms that Recursive Binary Search has logarithmic running time. Increasing the dataset from 1,000 to 1,000,000 records increases the search depth from approximately 10 to approximately 20 levels, rather than increasing it by a factor of 1,000.

The recurrence describes the search phase. If the data must first be sorted, the additional preprocessing cost must be included separately.

**TASK 8 — Implement and Test the Algorithms.**

Python is used because it provides built-in support for lists and dictionaries, allowing the three search strategies to be implemented clearly.

The following implementation uses registration numbers generated in ascending order. Linear Search and Hash-Table Search can use these records without relying on their order; Binary Search relies on the ordering.

**8.1 Python implementation**

```
import time import statistics



def linear_search(records, target):
    for i, registration in enumerate(records):
if registration == target:
            return          i
return -1



def recursive_binary_search(records, target, low=0, high=None):
if high is None:
        high = len(records) - 1


    if    low    >    high:
return -1


    mid = (low + high) // 2
```

```
    if records[mid] == target:
        return mid

    if target < records[mid]:
        return recursive_binary_search(
records, target, low, mid - 1
        )

    return recursive_binary_search(
records, target, mid + 1, high
    )


def build_hash_table(records):     return
{registration: i             for i, registration in
enumerate(records)}


def      hash_search(hash_table,      target):
return hash_table.get(target, -1)


def generate_records(n):
    return [f"REG{i:07d}" for i in range(n)]


def benchmark(function, repetitions=5):
    measurements = []

    for _ in range(repetitions):         start =
time.perf_counter()         function()         end =
```

```
time.perf_counter()
measurements.append((end - start) * 1000)

    return statistics.median(measurements)


def run_tests():
    sizes = [1_000, 10_000, 100_000, 1_000_000]

    for n in sizes:
        records = generate_records(n)
target = records[-1]         missing =
"REG99999999"

        # Correctness tests          assert
linear_search(records, records[0]) == 0          assert
linear_search(records, target) == n - 1          assert
linear_search(records, missing) == -1

        assert               recursive_binary_search(
records, records[0]
        ) == 0         assert
recursive_binary_search(
records, target         ) == n - 1
assert recursive_binary_search(
records, missing
        ) == -1

        # Measure hash-table construction separately
build_ms = benchmark(              lambda:
build_hash_table(records), 3
        )
```

```
    hash_table = build_hash_table(records)

    assert hash_search(hash_table, records[0]) == 0
assert hash_search(hash_table, target) == n - 1      assert
hash_search(hash_table, missing) == -1

    # Search timings: target at the last position
linear_ms = benchmark(          lambda:
linear_search(records, target), 3
    )
    binary_ms = benchmark(
lambda: recursive_binary_search(
records, target
        ), 100
    )
    hash_ms  =  benchmark(                  lambda:
hash_search(hash_table, target), 1000
    )

    print(f"\nDataset size: {n:,}")
print(f"Linear Search: {linear_ms:.6f} ms")
print(f"Binary Search: {binary_ms:.6f} ms")
print(f"Hash construction: {build_ms:.6f} ms")
print(f"Hash lookup: {hash_ms:.6f} ms")



if __name__ == "__main__":
   run_tests()
```

**8.2 Testing procedure**

The experiment uses the following procedure:

1. Generate datasets containing 1,000, 10,000, 100,000 and 1,000,000 unique registration numbers.

2. Use the same generated records for all three algorithms.
3. Verify successful searches for the first and last registration numbers.
4. Verify unsuccessful searches using a registration number absent from the dataset.
5. Measure hash-table construction separately from lookup.
6. Measure search time using time.perf_counter().
7. Repeat measurements and report the median to reduce the effect of isolated timing fluctuations.
8. Record input size, operation count and execution time.

For the worst-case Linear Search experiment, the target is the last registration number. This forces the algorithm to inspect every record.

**Experimental limitation:** The records are generated in sorted order for convenience. This means the timing experiment does not include the cost of sorting an initially unsorted dataset. A separate sorting benchmark would be required to measure that cost directly.




## TASK 9 — Record Experimental Performance.

### 9.1 Experimental environment and results

The following are sample measurements obtained by running the Python search functions in an execution environment. They are machine- and runtime-specific observations, not universal performance guarantees.

The measured input consists of unique, fixed-width registration-number strings. Linear Search targets the last record; Binary Search targets the last record in the sorted list; Hash-Table Search uses a previously constructed dictionary.

| Records | Linear Search (ms) | Binary Search (ms) | Hash construction (ms) | Hash lookup (ms) |
|---|---|---|---|---|
| 1,000 | 0.0333 | 0.00143 | 0.0819 | 0.000093 |
| 10,000 | 0.3780 | 0.00193 | 2.6882 | 0.000094 |
| 100,000 | 3.7691 | 0.00226 | 19.9659 | 0.000091 |
| 1,000,000 | 49.1816 | 0.00268 | 295.9956 | 0.000102 |

The times are rounded for presentation. Hash-table construction is listed separately because it is a preprocessing operation rather than part of a single lookup.

### 9.2 Operation counts

For Linear Search, the target is at the last position, so the comparison count is exactly (n).

For Binary Search, the number of midpoint examinations for this target is 10, 14, 17 and 20, respectively. These counts reflect the actual search path and are not necessarily the maximum for every possible input size.

| Records | Linear comparisons | Binary midpoint examinations | Expected hash lookup behaviour |
|---|---|---|---|
| 1,000 | 1,000 | 10 | Approximately constant |
| 10,000 | 10,000 | 14 | Approximately constant |
| 100,000 | 100,000 | 17 | Approximately constant |
| 1,000,000 | 1,000,000 | 20 | Approximately constant |

Hash-table entry examinations depend on the hash function and the distribution of keys. They should be measured separately if an exact operation count is required.

### 9.3 Graphical presentation

The following Python code generates a plot from the recorded search timings. A logarithmic vertical axis makes the differences between algorithms visible.

```
import matplotlib.pyplot as plt


sizes = [1000, 10000, 100000, 1000000]


linear_ms = [0.033275, 0.377994, 3.769147, 49.181628] binary_ms
= [0.0014285, 0.001931, 0.0022605, 0.002678] hash_ms =
[0.000093, 0.000094, 0.000091, 0.000102] plt.figure(figsize=(9,
5)) plt.plot(sizes, linear_ms, marker="o", label="Linear Search")
plt.plot(sizes, binary_ms, marker="o", label="Binary Search")
plt.plot(sizes, hash_ms, marker="o", label="Hash-Table Search")


plt.xscale("log") plt.yscale("log")
plt.xlabel("Number of student records")
plt.ylabel("Search time (milliseconds)")
plt.title("Search Performance versus Dataset Size")
```

plt.legend() plt.grid(True, which="both")

plt.tight_layout() plt.show()

The plotting code uses the measurements above. If the implementation is run on a different computer, the graph should be regenerated using the new measurements rather than treating these timings as universal.

**TASK 10 — Compare Theory with Experimental Behaviour.**

**10.1 Linear Search**

The theoretical worst-case comparison count is (n). Experimentally, the measured time increases from approximately 0.033 ms at 1,000 records to 49.18 ms at one million records.

The increase is consistent with linear growth: the algorithm must examine every record when the target is last.

The measured time does not increase by exactly the same factor as (n) because execution time also depends on interpreter overhead, CPU scheduling, memory access and timing variability.

**10.2 Recursive Binary Search**

The theoretical search complexity is (\Theta(\log n)). The number of midpoint examinations increases from 10 to 20 as the dataset grows by a factor of 1,000.

The observed execution time remains very small, increasing from approximately 0.00143 ms to 0.00268 ms in this experiment. The modest increase is consistent with logarithmic growth, although the absolute timings are affected by measurement noise and runtime overhead.

The experimental data should not be interpreted as proving the exact asymptotic bound. The mathematical analysis establishes the bound; the measurements provide supporting evidence.

**10.3 Hash-Table Search**

The expected lookup complexity is (\Theta(1)) under the stated assumptions. The measured lookup time remains approximately 0.00009–0.00010 ms across the tested sizes.

This is consistent with expected constant-time lookup. However, the hash-table construction time increases with dataset size, from approximately 0.082 ms to 296 ms.

The experiment illustrates why preprocessing must be reported separately: an index may require substantial construction time even though subsequent lookups are very fast.

**10.4 Reasons for theoretical and experimental differences**

Several factors can influence observed execution times:

- **Interpreter overhead:** Python function calls and loop execution contribute to total runtime.
- **Memory allocation:** Constructing a dictionary requires memory allocation and internal table management.

- **Cache effects:** Different dataset sizes may interact differently with CPU caches and memory.
- **Hash collisions:** Lookup cost varies with key distribution and table load factor.
- **Measurement noise:** Very short operations may be close to the timer's practical measurement resolution.
- **Input conditions:** A first-element Linear Search and a last-element Linear Search have very different operation counts.

The main conclusion is that the observed trends agree with the theoretical models: Linear Search scales linearly, Binary Search scales logarithmically, and Hash-Table Search has approximately constant expected lookup time.

**TASK 11 — Recommend and Defend the Final Design.**

**11.1 Recommendation**

For a university that frequently retrieves records using unique registration numbers, **Hash-Table Search is the recommended primary exact-match lookup strategy**, provided that the system can accommodate the additional index memory and maintain the index correctly.

**11.2 Mathematical justification**

The expected complexity comparison is:

- Linear Search: (\Theta(n)) per worst-case lookup.
- Binary Search: (\Theta(\log n)) per lookup after sorting.
- Hash-Table Search: (\Theta(1)) expected lookup after construction.

For one million records, Linear Search may examine one million keys, whereas Binary Search requires approximately 20 midpoint examinations. Hashing generally examines a small number of entries, subject to the hash implementation and collision behaviour.

**11.3 Experimental justification**

In the measured experiment, Linear Search required approximately 49.18 ms for a worst-position lookup on one million records. Binary Search required approximately 0.00268 ms, and Hash-Table Search approximately 0.00010 ms after preprocessing.

These measurements favour hash-based lookup for frequent exact-match searches. They are specific to the experimental environment and should be validated on the university's intended production hardware.

**11.4 Preprocessing and trade-offs**

Hashing requires expected (O(n)) construction time and (O(n)) additional space. If records change, the index must be updated to avoid returning stale results.

Binary Search is a reasonable alternative when memory is limited, sorted order is already maintained or ordered queries are also required. Linear Search remains appropriate for very small datasets or occasional searches where building an index would not be worthwhile.

### 11.5 Final design

The recommended design is to maintain a hash index keyed by registration number and use it for exact-match retrieval. A separate sorted index can be maintained if the university also requires ordered reports or range queries.

The recommendation is based on the combination of expected lookup efficiency, measured performance, construction cost, memory requirements and the exact-match nature of the university's search workload—not merely on selecting the algorithm with the smallest Big-O expression.

## TASK 12 — DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN.

### A. Impact Analysis.

### Changed requirements

The revised scenario specifies:

- One million student records.
- 50,000 search requests per hour.
- New records added only once per day.

This changes the workload from a general search problem across several possible dataset sizes to a high-frequency lookup problem against a large, relatively stable dataset.

### Why search frequency matters

The number of requests per hour is

$$
q=50{,}000.
$$

The average request rate is

$$
\frac{50{,}000}{3{,}600}
\approx 13.89 \text{ searches per second}.
$$

If every request used worst-case Linear Search on one million records, the theoretical maximum comparison workload would be

$$
50{,}000\times1{,}000{,}000
=50{,}000{,}000{,}000
$$

or 50 billion key comparisons per hour.

This corresponds to approximately 13.89 million comparisons per second on average.

By comparison, Binary Search would need at most about 20 midpoint examinations per search for a dataset of this size, or approximately one million midpoint examinations per hour.

Hashing offers expected constant-time lookup, making it particularly suitable for a high volume of exact-match requests.

**Why infrequent updates matter**

Because new records are added only once per day, the university can construct or refresh the index during a scheduled update period. It does not need to rebuild the index for every search.

The low update frequency therefore makes the initial construction cost easier to amortize over the many lookups performed between updates. **B. Evaluation of the Original Design.**

The original recommendation was Hash-Table Search. It remains technically appropriate under the changed workload.

The measured construction time for one million records was approximately 296 ms in the experimental environment, while an individual hash lookup took approximately 0.00010 ms. Actual production times will depend on the hardware, database implementation, memory usage and concurrency.

The hash table should not be reconstructed for every search. Instead, it should be built once and retained as a maintained index.

For comparison, Binary Search is also viable if the records are sorted and kept sorted. Its logarithmic lookup cost is substantially better than Linear Search, but it is expected to perform more key comparisons than a well-behaved hash index for exact-match retrieval.

Linear Search is not appropriate as the primary production strategy for one million records and 50,000 hourly searches because its worst-case work grows proportionally to the entire dataset.

**C. Adaptation or Redesign.**

**Decision: Retain and refine the hash-based design**

The recommended strategy is to retain Hash-Table Search, with the following operational refinements:

1. Build the hash index from the authoritative student database during initialization or the daily update process.
2. Use registration numbers as unique keys.
3. Apply a suitable hash function, collision-resolution strategy and resizing policy.
4. Insert daily additions into the existing index or rebuild the index in a controlled batch, depending on the implementation.
5. Verify that the index and the authoritative database remain consistent.
6. Measure lookup latency and memory consumption under representative concurrent workloads.

**Revised complexity**

Let (n=1{,}000{,}000) and (q=50{,}000) searches per hour.

With a daily build and expected constant-time lookup,

[
T_{\text{hour}}(n,q)=\Theta(n)+\Theta(q)
]

if the construction cost is allocated to the hour in which the build occurs. Across the full day, the total expected work is

[
T_{\text{day}}(n,q_d)=\Theta(n)+\Theta(q_d),
]

where (q_d) is the total number of searches during that day.

If 50,000 searches occur in each of 24 hours, then

[
q_d=24\times50{,}000=1{,}200{,}000.
]

The expected daily work is therefore proportional to the one-time construction cost plus the number of daily lookups, rather than the product of the number of records and the number of searches.

**D. Evidence-Based Defence.**

The decision to retain and refine Hash-Table Search is supported by four considerations.

**1. Mathematical analysis**

Expected lookup time is (\Theta(1)), compared with (\Theta(\log n)) for Binary Search and (\Theta(n)) for Linear Search. Hash-table construction requires expected (\Theta(n)) work and additional memory.

**2. Experimental evidence**

The measured one-million-record search times were approximately:

- Linear Search: 49.18 ms.
- Recursive Binary Search: 0.00268 ms.
- Hash-Table Search: 0.00010 ms after index construction.

The hash-table build took approximately 296 ms in the same experimental environment. These figures are illustrative measurements from that environment, not guaranteed production performance.

**3. Workload characteristics**

The system processes approximately 13.89 searches per second on average. A persistent index avoids repeating preprocessing work for each request.

**4. Update frequency and trade-offs**

Daily additions allow the system to update the index in batches or incrementally. The principal trade-offs are memory consumption, collision management, index consistency and the need to preserve correct results during updates.

**Final decision**

**Retain the hash-based lookup strategy and optimize its operational management.** It matches the university's unique-key search requirement, offers expected constant-time lookups and allows its construction cost to be amortized over a large number of requests. The decision should be validated with production-scale load tests, including concurrent requests, unsuccessful searches, daily updates and memory measurements.